018187975

Homework 2 3D Object Detection

INTRODUCTION

3D object detection from LiDAR data is a fundamental perception task in autonomous driving pipelines. Accurate localization and classification of objects in 3D space directly impact downstream modules such as multi-object tracking, motion prediction, and planning. However, running state-of-the-art detection models requires careful model configuration, dependency management, and efficient execution pipelines for large-scale datasets such as KITTI and nuScenes.

This project implements a reproducible inference and profiling workflow for PointPillars using the MMDetection3D framework on the SJSU EdgeAI lab server. Our goals are: (1) run inference on large LiDAR datasets using pre-trained MMDetection3D checkpoints; (2) save complete inference artifacts for later visualization and benchmarking; (3) evaluate performance along throughput and memory efficiency dimensions; and (4) resolve major system-level challenges related to CUDA, PTX compatibility, and Numba JIT evaluation kernels. The resulting system enables accurate 3D detection while providing quantifiable performance baselines for continued research on efficient LiDAR perception.

METHODOLOGY

Our methodology integrates model configuration, execution, artifact persistence, and performance evaluation into a modular pipeline. We use MMDetection3D model configurations provided by OpenMMLab and pre-trained weights for PointPillars. A custom inference wrapper executes batches of LiDAR point clouds, measures throughput and memory consumption, and logs results in machine-readable formats.

The pipeline interacts with the lab server's GPU environment through conda environments configured for CUDA, PyTorch, mmcv, and mmdet3d. To ensure reproducibility, performance scripts export JSON predictions and PLY formatted point clouds for Open3D-based offline visualization. Runtime metadata such as FPS, GPU memory footprint, and batch size statistics are logged for analysis.

EXPERIMENTAL SETUP

All experiments were executed on the SJSU EdgeAI server backend configured with NVIDIA GPUs and CUDA drivers. We maintained two file trees:

- Base repository (read-only):
  /home/student/018187975/LiDAR-3D-Detection
- Working repository (private copy):
  /home/JaiPrathiksoda/Lambda

Environment:

- OS: Ubuntu 20.04 LTS
- GPU: Tesla T4
- CUDA: 11.x

- Python Conda environment: PyTorch + mmcv + mmdetection3d dependencies

Execution command (example):

```
python run_inference_save_artifacts.py \
  --config checkpoints/pointpillars_hv_secfpn_8xb6-160e_kitti-3d-car.py \
  --checkpoint checkpoints/hv_pointpillars_*.pth \
  --pcd-dir data/kitti/training/velodyne \
  --out-dir outputs/pointpillars_kitti \
  --dataset kitti \
  --max-samples 50
```

MODELS AND CHECKPOINTS

The following MMDetection3D pretrained weights were used:

Model	Dataset	Backbone	Purpose
PointPillars	KITTI	SECOND + 2D CNN head	LiDAR object detection baseline

The inference scripts initialize models using mmengine runners and MMDetection3D config files. No training was performed, only inference and evaluation using official pretrained checkpoints to maintain reproducibility and isolate inference-time performance.

METRICS

We evaluate two main performance categories:

1. **Detection Metrics**

- mAP (3D bounding box IoU thresholds)
- Class-wise precision and recall

2. **System Performance Metrics**

- **Throughput** (frames per second per batch and per model)
- **Latency** (inference time per frame)
- **GPU Memory Utilization**
- **Peak and steady-state memory profile**
- **PTX kernel compilation overhead**

These metrics characterize not only detection accuracy but operational feasibility for real-time AV systems.

RESULTS

3.1 Performance Summary

Experiments demonstrate that PointPillars achieves robust frame-level detection while sustaining real-time throughput on the EdgeAI GPU. On KITTI LiDAR frames, inference averaged near real-time processing speeds while maintaining stable GPU memory usage. Saved inference artifacts indicate consistent 3D bounding box prediction structure and calibration alignment.

3.2 Throughput Analysis

Throughput was measured using aggregated timing from batch executes. The average processing rate per clip remained within expected OEM benchmarks for PointPillars, confirming that optimized CUDA execution pipelines within MMDetection3D were successfully leveraged.

Key observations:

- FPS stability improved when Numba PTX compatibility fixes were applied.
- First-run latency increased slightly due to just-in-time kernel compilation.

3.3 Memory Utilization

Peak GPU memory was recorded before and during inference loads. For PointPillars, memory usage remained within the constraints expected for single-GPU deployments, although small spikes were observed while caching voxelization intermediate tensors and CUDA JIT compilation caches.

3.4 Memory Profile – nuScenes PointPillars

nuScenes scenes showed higher memory pressure due to:

- denser scans
- higher sweep fusion
- multi-frame preprocessing

This validates that memory-aware batching and incremental processing strategies are required when scaling object detection to full sequences.

ANALYSIS

4.1 Key Findings

- GPU throughput and memory characteristics are strongly influenced by preprocessing and voxel encoding rather than detection head complexity.
- PTX/Numba incompatibility can block evaluation entirely unless properly configured.
- Artifact storage ensures full traceability and offline visualization reproducibility.

4.2 Architecture Comparison

Although only PointPillars was evaluated experimentally, system logs and literature indicate that pillarization-based architectures perform inference significantly faster than voxel-based or transformer-based backbones, confirming the suitability for real-time AV workloads.

5 Limitations

- Experiments rely only on pre-trained models and do not include retraining or fine-tuning.
- Throughput measurements do not capture multi-camera fusion settings.
- nuScenes evaluation was constrained to limited sample subsets due to storage barriers on the shared server.

TECHNICAL CHALLENGES

6.1 CUDA / Numba PTX Version Mismatch

During evaluation, Numba invoked CUDA JIT kernels to compute rotated IoUs. The server's GPU driver supported an older PTX target than the Numba CUDA backend was attempting to compile. This resulted in:

```
CUDA_ERROR_UNSUPPORTED_PTX_VERSION
numba.cuda.cudadrv.driver.CudaAPIError
```

We resolved this by configuring Numba to compile kernels targeting a compatible compute capability via environment variables and persistent configuration files. The fix preserved GPU acceleration and avoided CPU fallback execution, maintaining runtime feasibility for large-scale detection evaluation.

CONCLUSION

This project demonstrates a complete reproducible LiDAR 3D inference pipeline for object detection using MMDetection3D's PointPillars model. We successfully executed inference, saved complete artifacts, and evaluated system-level performance including throughput and memory footprints. Critical failures arising from CUDA PTX mismatches were resolved through targeted Numba configuration, enabling full GPU execution on the lab environment. The profiling and evaluation results provide reliable baselines for future development of more advanced or efficient perception architectures, sensor fusion strategies, or multi-stage tracking pipelines.